

Documento Completo del Proyecto - SGPR

Proyecto: Sistema de Gestión de Permisos y Reportes (SGPR)

Desarrollado bajo: Metodología SCRUM

Responsable del desarrollo: Una sola persona (análisis, diseño, desarrollo, pruebas e implementación)

Periodo de Sprint (documentado): 01/04/2026 – 30/06/2026

---

## 1. Visión

Ser la plataforma de referencia para la gestión y control de permisos laborales en la organización, ofreciendo reportes precisos, exportes confiables y controles de auditoría que faciliten la toma de decisiones y el cumplimiento normativo.

## 2. Misión

Proveer una aplicación web segura y fácil de usar que centralice la gestión de solicitudes de permisos y reposos, automatice cálculos de días laborables, genere reportes visuales y exportables, y mantenga trazabilidad de todas las acciones mediante auditoría.

## 3. Objetivo general

Diseñar, desarrollar e implementar un sistema que permita registrar, revisar y reportar solicitudes de permiso y reposo, garantizando integridad, confidencialidad y trazabilidad de la información.

## 4. Objetivos específicos

- Implementar modelos de datos para Trabajador, Solicitud y Auditoría con cifrado para campos sensibles.
- Desarrollar interfaces para registro y gestión de solicitudes.
- Incorporar visualización de estadísticas con Chart.js y exportes PDF/XLSX que reproduzcan los gráficos.
- Registrar auditoría de acciones críticas y mantener logs seguros.
- Desplegar la aplicación en servidor (Gunicorn + Nginx) para reproducibilidad.

## 5. Propósito y justificación

Centralizar la gestión de permisos reduce errores administrativos, agiliza procesos y provee evidencia documental ante auditorías. El sistema sustituye procesos manuales y facilita la generación de reportes para RRHH y administración.

## 6. Alcance del proyecto

Incluye:

- Módulo Trabajadores (perfil y vínculo con usuario Django).
- Módulo Solicitudes (creación, edición, validación, cálculo automático de días laborables y adjuntos).
- Panel de Estadísticas (gráficos, filtrado, exportes PDF/XLSX).
- Módulo Auditoría (registro de acciones con detalles cifrados).
- Despliegue en servidor (configuración básica: Gunicorn + Nginx).

No incluye: integración SSO externa, migraciones masivas automatizadas ni sistemas de nómina.

## 7. Descripción del proyecto

Aplicación web Django. Frontend con plantillas y Chart.js para gráficos. Exportes PDF intentan capturar el canvas cliente (canvas.toDataURL); si falla (canvas tainted) se genera gráfico en servidor con matplotlib y se ensambla el PDF con ReportLab y Pillow. Campos sensibles (ej. cédula, motivo) se almacenan cifrados usando EncryptedTextField.

---

## 8. Stakeholders (Interesados)

- Administrador del sistema
- Recursos Humanos
- Trabajadores (usuarios finales)
- Auditoría interna
- Equipo de soporte/IT

---

## 9. Historias de usuario / Casos de uso

- HU-001: Registrar trabajador.
- HU-002: Crear solicitud de permiso con adjunto.
- HU-003: Visualizar y filtrar solicitudes.
- HU-004: Aprobar o rechazar solicitud con observaciones.
- HU-005: Generar reporte PDF/XLSX del trabajador con gráficos.
- HU-006: Consultar registro de auditoría por usuario/tabla.

Para cada HU se documenta: actor, precondition, flujo principal, excepciones, postcondición.

---

## 10. Requerimientos funcionales (selección principal)

- RF1: CRUD para Trabajadores.
- RF2: CRUD para Solicitudes con validación de fechas y adjuntos.
- RF3: Cálculo automático de días continuos y días laborables.
- RF4: Visualización de estadísticas con filtros.
- RF5: Exportar estadísticas a PDF y XLSX incluyendo gráficos.
- RF6: Registro de auditoría para acciones críticas.
- RF7: Autenticación y control de permisos basados en Django.

## 11. Requerimientos no funcionales

- RNF1: Seguridad: cifrado en BD para campos sensibles.
- RNF2: Rendimiento: respuesta aceptable (<2s) en operaciones básicas con datasets pequeños.
- RNF3: Disponibilidad y despliegue reproducible en servidor (Gunicorn + Nginx).
- RNF4: Mantenibilidad: código documentado y pruebas básicas.
- RNF5: Portabilidad: soporte PostgreSQL/SQLite.

---

## 12. Backlog del Producto (epicas y user stories principales)

- Épica: Gestión de Trabajadores
- US-101: Implementar modelo Trabajador y enlace con User
- US-102: Formulario y listados
- Épica: Solicitudes
- US-201: Crear Solicitud, validaciones, adjuntos
- US-202: Calculo de días y reglas de feriados
- Épica: Reportes y Estadísticas
- US-301: Chart.js en UI
- US-302: Export PDF (cliente+servidor)
- US-303: Export XLSX
- Épica: Auditoría
- US-401: Registrar acciones con detalles cifrados

---

## 13. Sprint Backlog (Periodo: 01/04/2026 → 30/06/2026)

Este periodo abarca 3 meses. Se propone dividir en sprints de 2 semanas. Resumen:

- Sprint 1 (01/04 – 14/04): Modelos y configuración base
- Implementar modelos: Trabajador, Solicitud, Auditoria
- Configuración Django y entorno virtual básico
- Pruebas unitarias de modelos
- Sprint 2 (15/04 – 28/04): Formularios y CRUD
- Formulario Solicitud y validaciones

- Adjuntos y validación de extensiones
- Encriptación de campos
- Sprint 3 (29/04 – 12/05): Estadísticas UI
- Integrar Chart.js, filtros, paleta de colores
- Ajustes visuales y pruebas manuales
- Sprint 4 (13/05 – 26/05): Exportes PDF/XLSX
- Captura canvas client-side y envío base64
- Implementar fallback server-side (matplotlib+ReportLab)
- Pruebas de exportes y ajustes de tamaño/espaciado
- Sprint 5 (27/05 – 09/06): Auditoría y seguridad
- Registro de auditoría en acciones críticas
- Revisar cifrado y gestión de claves (env vars)
- Sprint 6 (10/06 – 23/06): Deploy y pruebas finales
- Ajustes de despliegue en servidor, scripts backup
- Pruebas de integración y corrección de bugs
- Buffer/Entrega (24/06 – 30/06): Documentación final, manuales, retrospectiva y entrega al cliente.

(Tareas detalladas y estimaciones se registran en el backlog del proyecto).

---

#### 14. Acta de seguimiento de reuniones del Sprint (plantilla y ejemplo)

Acta – Reunión de seguimiento (Daily/Weekly)

- Fecha: 2026-05-05
- Participantes: Desarrolladora (responsable única)
- Objetivos del día/sprint: Finalizar exportes PDF; corregir tamaño de gráficos
- Avances: Captura cliente funciona en navegador X; fallback en servidor implementado
- Bloqueos: Canvas tainted en navegadores con recursos cross-origin
- Acuerdos: Enviar paleta de colores desde frontend al backend en próxima iteración
- Próximos pasos: Ajustar márgenes en ReportLab; generar PDF de prueba

(Guardar actas en `/docs/actas/` con fecha en el nombre).

---

#### 15. Informe consolidado mensual de acciones (Ejemplos)

Abril 2026

- Implementación de modelos y CRUD básicos.
- Configuración inicial del entorno de despliegue en servidor.
- Pruebas unitarias de modelos.

Mayo 2026

- Desarrollo de panel de estadísticas y Chart.js.
- Implementación de export PDF con captura cliente y fallback servidor.
- Correcciones de UI y formato de reportes.

Junio 2026

- Auditoría y seguridad reforzadas.
- Tests de integración y despliegue en servidor.
- Documentación y manuales técnicos.

---

#### 16. Wireframes y diseño de interfaz

- Páginas principales: Login, Listado Trabajadores, Formulario Solicitud, Panel Estadísticas, Detalle Solicitud.
- Organización responsive con Bootstrap y componentes accesibles.
- Incluir wireframes en `/docs/wireframes/` (PNG/SVG).

---

#### 17. Diagrama de Base de Datos (MER)

Se incluye `erd.puml` en el repositorio. Para imagen generada usar PlantUML.

---

## 18. Interfaces de entrada y salida

### Entradas:

- Formularios HTML (crear/editar solicitudes y trabajadores)
- Archivos adjuntos (multipart)
- Parámetros de filtrado en panel de estadísticas (fechas, trabajador, tipo)

### Salidas:

- Vistas HTML con tablas y gráficos
- Exportes PDF y XLSX
- Registros de auditoría

---

## 19. Web Services básicos (APIs)

- `GET /api/solicitudes/` – listar solicitudes (filtros: trabajadorid, estado, fecha)
- `POST /api/solicitudes/` – crear solicitud (multipart)
- `GET /api/estadisticas/` – datos para graficas (JSON)
- `POST /api/export/pdf/` – generar PDF (recibe imágenes base64 o datos de fallback)

Asegurar autenticación y control de acceso en cada endpoint.

---

## 20. Base de datos (backup y restauración)

- Recomendación: usar `pgdump` para PostgreSQL y rotación diaria. Scripts en `/ops/backup.sh`.
- Mantener backups en un directorio seguro (`/backups/`) con retención mínima de 7 días.

---

## 21. Desarrollo e implementación de módulos (estado resumido)

- Trabajadores: implementado (modelos, vistas, templates)
- Solicitudes: implementado (validaciones, adjuntos, cálculos)
- Estadísticas: implementado (Chart.js, exportes)
- Auditoría: implementado (registro y almacenamiento cifrado)

Módulos probados localmente con `runserver`. (Despliegue en servidor probado según plan de lanzamiento).

---

## 22. Formularios digitales (estructura)

### Formulario Solicitud

- trabajadorid (select)
- tipo (select)
- fechainicio (date)
- fechafin (date)
- motivo (textarea)
- adjunto (file)

Validaciones: fechas coherentes, adjunto extensión permitida, motivo obligatorio.

---

## 23. Módulos integrados y pruebas de instalación

- Instalar dependencias: `pip install -r requirements.txt`.
- Levantar servicios en producción: iniciar el servidor de aplicaciones (p. ej. Gunicorn) y configurar Nginx como proxy inverso.
- Verificar acceso en `http://localhost:8000`.

---

#### 24. Código y archivos fuentes

- Repositorio contiene: `sgpralquitrana/` (app Django), `templates/`, `static/`, `erd.puml`.
- Código documentado en español; no contiene credenciales.

---

#### 25. Informe de testeo / pruebas realizadas

- Pruebas unitarias de modelos y validaciones.
- Pruebas funcionales manuales: creación de solicitudes, aprobación, exportes, auditoría.
- Casos especiales: canvas tainted (verificado fallback), adjuntos inválidos rechazados.

---

#### 26. Manuales técnicos

- Manual de usuario (`/docs/manualusuario.md`) – Uso del sistema, generación de reportes.
- Manual de implementación (`/docs/manualimplementacion.md`) – Despliegue en servidor, restauración de backups.

---

#### 27. Documentos de retrospectiva final

- Lecciones aprendidas: definir paleta de colores consistente, manejar limitaciones de canvas, priorizar tamaño de payload base64.
- Recomendaciones: automatizar backups, pruebas en múltiples navegadores y entornos, documentar procesos operativos.

---

#### 28. Entregables y evidencia de instalación

- Código fuente y documentación en el repositorio.
- `erd.puml` con MER y explicación.
- Scripts de backup y despliegue en `/ops/`.
- Carpetas `/docs/manuals/`, `/docs/wireframes/`, `/docs/actas/` con archivos correspondientes.

---

#### 29. Firma y declaración

Declaro que este proyecto fue desarrollado bajo SCRUM y que la persona indicada (desarrolladora) ejecutó todas las actividades de análisis, diseño, desarrollo, pruebas e implementación.

Fecha de generación del documento: 2026-06-03

---

Fin del documento.